



# HAPPY: Hybrid Address-based Page Policy in DRAMs

Mohsen Ghasempour<sup>†</sup>, Aamer Jaleel<sup>\*</sup>, Jim D. Garside<sup>†</sup> and Mikel Luján<sup>†</sup>

<sup>†</sup>School of Computer Science, University of Manchester, UK    <sup>\*</sup>NVIDIA, Westford, MA, USA

## ABSTRACT

Memory controllers have used static page closure policies to decide whether a row should be left open, *open-page policy*, or closed immediately, *close-page policy*, after the row has been accessed. The appropriate choice for a particular access can reduce the average memory latency. However, since application access patterns change at run time, static page policies cannot guarantee to deliver optimum execution time. Hybrid page policies have been investigated as a means of covering these dynamic scenarios and are now implemented in state-of-the-art processors. Hybrid page policies switch between open-page and close-page policies while the application is running, by monitoring the access pattern of row hits/conflicts and predicting future behavior. Unfortunately, as the size of DRAM memory increases, fine-grain tracking and analysis of memory access patterns does not remain practical.

We propose a compact memory address-based encoding technique which can improve or maintain the performance of DRAMs page closure predictors while reducing the hardware overhead in comparison with state-of-the-art techniques. As a case study, we integrate our technique, HAPPY, with a state-of-the-art Intel-adaptive monitor (e.g. part of the Intel Xeon X5650) and a traditional Hybrid page policy. We evaluate them across 70 memory intensive workload mixes consisting of single-thread and multi-thread applications. The experimental results show that using the HAPPY encoding applied to the Intel-adaptive page closure policy can reduce the hardware overhead by 5× for the evaluated 64 GB memory (up to 40× for a 512 GB memory) while maintaining the prediction accuracy.

## CCS Concepts

•Computer systems organization → Architectures;

## Keywords

Memory Systems, DRAM, Page Closure Policy

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

MEMSYS 2016 October 3–6, 2016, Washington, DC, USA

© 2016 Copyright held by the owner/author(s). Publication rights licensed to ACM. ISBN 978-1-4503-4305-3...\$15.00

DOI: <http://dx.doi.org/10.1145/2989081.2989101>

## 1. INTRODUCTION

The performance of DRAM is sensitive to the memory access pattern of the running applications. Traditionally DRAM controllers have used a static row-buffer access policy, either *open-page* or *close-page*, to decide whether a row should be left open or closed immediately after their access. For workloads with high locality of accesses open-page works best since the target row is already open and multiple accesses to that row can be serviced with one activation. However, for workloads with more random memory accesses, close-page is a better option. In this case a row will be closed immediately after a memory access so the next memory request within the same bank does not need to wait for the precharge process of the open row. Moreover, neither the open-page nor close-page policy can deliver the ‘best’ execution time for all the workloads due to the dynamic nature of memory accesses. In this situation a *hybrid-page* policy, which is a mixture of open-page and close-page, is more desirable.

Different techniques have been proposed in the literature to select between open-page and close-page in DRAM memory controllers. *Access-based techniques* are those that monitor and keep a history of the row hits and row misses at different granularity in DRAMs to make a prediction of the future page closure policy. On the other hand, *time-based techniques* focus on predicting the optimum time that a row can be left open. In general, these techniques rely on predictors that monitor the number of accesses per row, the number of row hits or row misses, the time between hits or misses, etc. to predict the open-page or close-page for each row in DRAM. Intel included in the Xeon X5650 two time-based techniques.

As the size of DRAM is increasing with Data Analytic applications, having a fine-grain prediction and monitoring scheme is inefficient and not scalable. On the other hand going toward the coarse-grain schemes reduces the accuracy of the prediction. A key challenge for page-closure techniques is to balance the hardware overhead and the prediction accuracy. The trend towards keeping entire databases in DRAMs, such as RAMCloud (e.g. 64 TB of DRAMs) [1] or Facebook using 150 TB of DRAMs with memcache [18], turns the scalability issue into a critical problem for future systems.

Our contribution is a scalable and compact memory address-based encoding technique, called *HAPPY*, that can be employed in DRAM memory controllers. HAPPY is an efficient encoding that reduces the cost of implementation of existing page closure techniques while maintaining the prediction

accuracy of the original implementation. As case studies, we show how to integrate HAPPY with a state-of-the-art implementation – the Intel-adaptive open-page policy employed by Intel – and with a traditional hybrid-page. We evaluate HAPPY and existing techniques across 70 memory intensive workload mixes consisting of single-thread and multi-thread applications. The experimental results show that using the HAPPY memory address-based encoding applied to the Intel-adaptive page policy can reduce the hardware cost of implementation by 5× for the evaluated 64 GB memory while maintaining the prediction accuracy. In other words, we can achieve similar, or better, performance as existing high performance industry and academic techniques while requiring less hardware overhead.

## 2. BACKGROUND AND MOTIVATION

**DRAM Structure:** Figure 1 presents a high-level structure of a typical DRAM organization. A DRAM device (Figure 1a) consists of multiple banks, each of which includes a data array and a sense amplifier or *row buffer*. The data array is a matrix of rows and columns comprising the storage cells. The basic operation of DRAMs requires that to access a specific cell within a bank the entire row (e.g. 1 KB data) has to be moved into the row buffer. Then, read or write operations can be performed on the data stored in the row buffer. Although the banks within a DRAM device can be accessed in parallel, since they share the communication bus only one bank at the time can transfer data out of the DRAM device. Each DRAM device typically supports read/write operations of 4-16 bits per memory request depending on the DRAM model. To support the required bandwidth, multiple DRAM devices work in parallel within a Rank (Figure 1b). In modern DRAMs, 64-bit data can be read/written per cycle and typically a burst of size 4 or 8 is supported by these modules to fill a full cache-line [13, 12].

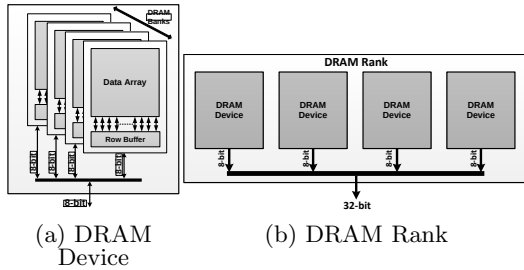


Figure 1: DRAM Structure.

**DRAM Basic Operation:** To perform a read or write operation, the target row has to be opened first using an *activation* command which transfers a row to the row buffer, imposing a delay  $t_{RCD}$ . When the row is in the row buffer, a read or write command can be issued with a delay  $t_{CL}$ . Considering the internal structure of DRAMs, only one row can be processed at a time. Thus, to access to a different row (within the same bank), the open row has to be closed first using the *precharge* command with a delay  $t_{RP}$ . This command prepares the row buffer to accept the new row. Considering the basic operation of DRAMs, each memory request can be classified into one of the following three categories depending on the status of the bank to be accessed: *page-hit*, *page-miss* or *page-empty*.

A page-hit is defined as a read/write operation to an open row within a bank. In this situation there is no need to use an activation command and the memory request can be serviced immediately. A page-miss is defined as a read/write operation to a different row than the open row within a bank. In this situation the open row must first be closed before accessing the second row. Finally, a page-empty is defined as a read/write command to a bank that has no open row in the row buffer. In this case an activation command is required to open the target row. Page-misses are the most expensive memory request while the page-hits are the cheapest to service. Page-empties are cheaper than the page conflicts but more expensive than the page-hits.

**DRAM Static Page Closure Policies:** DRAM memory controllers have a page closure policy, to alleviate the effect of page-misses on the memory system’s performance. The traditional schemes are the open-page and the close-page policy. A DRAM that uses the open-page policy would leave the last accessed row open in the row buffer to eliminate the activation cost of the next memory request to the same row. A DRAM that uses the close-page policy would close the row immediately after it has been accessed to eliminate the possibility of getting a page-miss for the next memory request [15]. In general, the open-page policy is more beneficial for the systems with high access locality whereas the close-page policy is more appropriate for systems with high entropy memory access. Table 1 presents the timing cost of page-hits and page-misses when using the static page closure policies.

| Page Policy      | Page Hit           | Page Miss                   |
|------------------|--------------------|-----------------------------|
| Open-Page        | $t_{CL}$           | $t_{RCD} + t_{CL} + t_{RP}$ |
| Close-Page       | $t_{RCD} + t_{CL}$ | $t_{RCD} + t_{CL}$          |
| Static Profiling | $t_{CL}$           | $t_{RCD} + t_{CL}$          |

Table 1: Cost of Page-hits and Page-misses when using different page closure policies.

**Motivation:** Figure 2 depicts the normalized execution time of all the workloads (to open-page policy) evaluated in this paper using open-page and close-page policy. The results show that around 68% of workloads prefer the open-page policy while 32% of workloads deliver a better performance using the close-page policy. According to this figure, a memory system that employs the open-page policy can save up to 18%, in comparison with the close-page policy, when running ‘libquantum’ from SPEC benchmark and, at the same time, might lose up to 18% when running ‘tigr’ from BIOBENCH benchmark. Therefore, there is almost a 40% of performance variation in the system depending on the static page policy that a memory controller employs. This motivates the development of dynamic page policies that switch between open and close-page policy at run time based on the application access behavior. The *Static Profiling* presented in Table 1 shows the cost of page-hits and page-misses when the memory controller selects the *best* static page closure policy scheme for each workloads by static profiling of memory accesses. The static profiling provides a baseline to evaluate the performance of dynamic page closure policies discussed in this paper.

Motivated by such observations, hybrid-page closure policies emerged. This type of page policies uses various prediction algorithms to switch dynamically between open- and

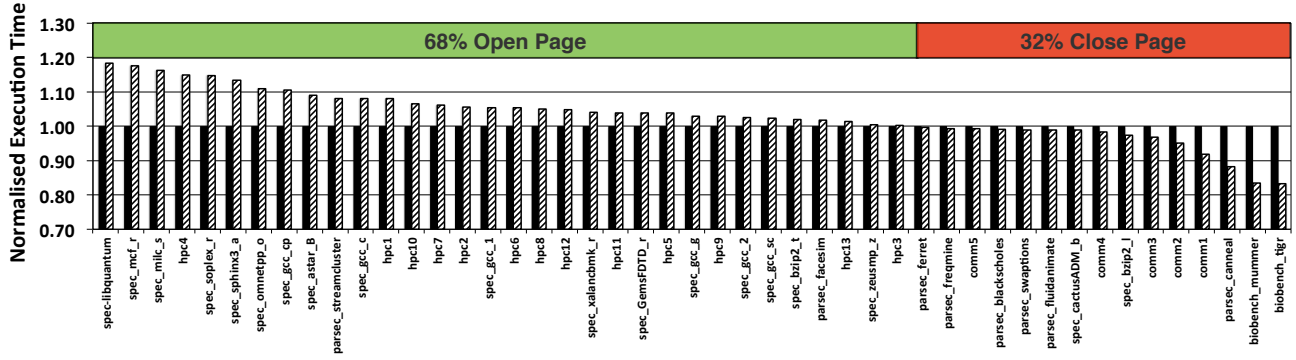


Figure 2: Performance of Static Page Policies for standard workloads.

close-page according to the application access behavior and improve performance. Prediction accuracy and scalability with increasing memory size are the two main constraints when designing such page closure predictors. For most techniques in the literature, there is a linear relationship between the DRAM memory size and the required resources to monitor the memory access pattern. Thus, as the memory size grows, the required cost of page closure policy predictor grows and, as a result, the on-chip memory controller complexity and area increase.

Overall, considering the scalability of emerging memory systems like HMCs, increasing the interest of using a large amount of DRAM instead of disk storage in servers and database analytic applications such as using 64 TB of DRAMs in RAMCloud [1, 19, 18] and using 150 TB of DRAMs by Facebook in memcache [18] all demand a scalable approach for efficient design. Section 3 introduces HAPPY as a compact encoding scheme to address the scalability problem of the page closure policy prediction technique for DRAM memory systems.

### 3. HAPPY:HYBRID ADDRESS-BASED PAGE POLICY

#### 3.1 HAPPY – Basic Principles

HAPPY is a compact memory address-based encoding built on the observation that there is a strong correlation between physical address bits and the internal structure of DRAMs. Understanding the basic operation of DRAMs shows that one of the first steps to access to the DRAM structure is the address mapping process. During this process, the physical address bits provided by a core are translated to the corresponding channel, rank, bank, row and column of a DRAM device using a fixed and pre-defined address-mapping algorithm. Having a fixed translation mapping creates a strong correlation between physical address bits and the DRAMs structure. It means that, if some useful information can be extracted from the physical address bits after translation, it is possible to extract the same information before this stage.

All the page closure algorithms proposed so far focus on monitoring the memory access behavior after the translation phase. In general, these techniques use different performance counters in a channel, bank or row basis to monitor, for example, page hits/conflicts, or the time that a row could

be kept open. HAPPY proposes a novel binary-encoding scheme with performance counters storing page closure history directly from the physical address bits. In the other words, HAPPY introduces one predictor per physical address bit to forecast the page closure policy of each row in the memory system according to the run time memory accesses.

Sections 3.2 and 3.3 show how HAPPY can be applied to the two most main page closure categories: access-based and time-based techniques. We illustrate one traditional and one state-of-the-art technique to demonstrate how the HAPPY encoding can be applied to different systems with different implementation characteristics. The methodology proposed in this paper can be applied to other aspects of a DRAM structure as well.

#### 3.2 HAPPY – Access-based Prediction

To demonstrate how HAPPY can be applied to access-based algorithms we select the traditional hybrid-page policy algorithm. This employs simple, saturating counters to monitor the memory access pattern behavior and dynamically switch between open- and close-page policy at run time. Figure 3 depicts the basic structure of such page closure policy predictors.

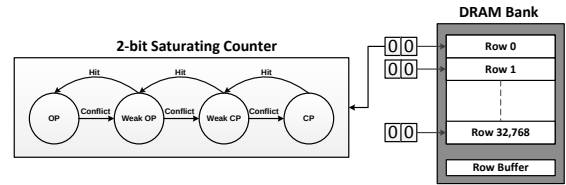


Figure 3: Basic structure of Hybrid Page Policy.

In this technique, one saturating counter (e.g. a 2-bit counter initialized to zero - open-page policy) is assigned to each row of a DRAM bank. Every time that a row-miss happens the corresponding counter is incremented. Whenever a row-hit occurs the counter is decremented. For each memory request, the accessed row's counter value determines the page closure policy; if the value is 0 or 1 the open-page policy is predicted and if the value of the counter is 2 or 3 the close-page policy is predicted. However, having a counter for each row in a DRAM device imposes a high area and power overhead to the memory system. For example, a 4 GB DRAM memory system with 1 channel, 2 ranks, 8 banks and 32,768

rows per bank, require 524,288 counters, which is not scalable (analysis presented in Section 5).

Figure 4 depicts the HAPPY implementation of a Hybrid page policy. The binary representation of the requested physical address is a pattern of zeros and ones. HAPPY dedicates two encoding counters per physical *address bit* location: one counter to monitor the position when its value is 0 and one to monitor it when it is 1. Training of these counters is similar to the original implementation of Hybrid; that means for every page conflict the counter corresponding to each physical address bit is incremented and for every page hit the same counter will be decremented. Considering the page-hits and page-misses happen in a per row basis then there is no need to monitor all the available physical address bits. Thus, the corresponding physical address bits to columns and cache-lines offset will not be used. This reduces even further the implementation costs of HAPPY.

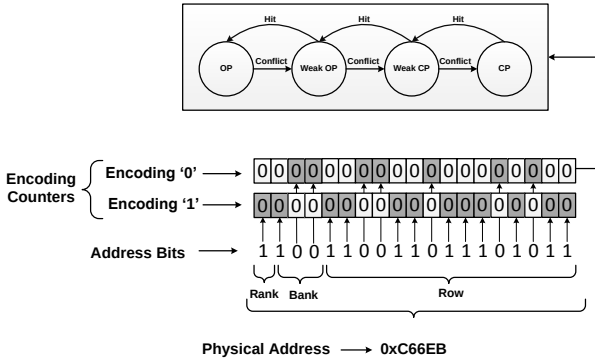


Figure 4: HAPPY implementation of Hybrid Page Policy.

Having done this, each physical address bit correlates with the possibility of getting a page hit/conflict depending on the value of that bit. Therefore, for a given physical address the possibility of getting page hit/conflicts can be calculated simply by considering all the bit values of the participant counters in the requested address and using one of the following techniques: Majority vote or Aggregation.

**Majority vote:** Figure 5 explains this scheme using a simple example. Each physical address bit has a counter which has its own standalone vote to choose an open- or close-page policy for the requested physical address. The page closure policy vote of each bit can be extracted by looking at the more significant bit of the saturating counters. If this bit is 0 there is an open-page policy vote and if the value is 1 there is a close-page policy vote. As the name implies, the final vote is determined by the majority vote across all the counters.

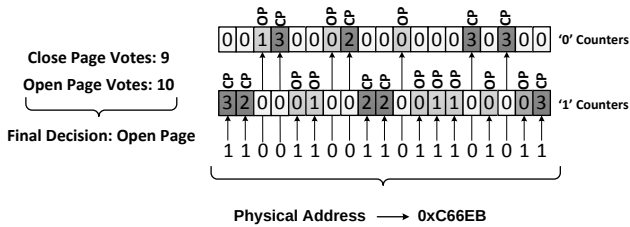


Figure 5: Example of HAPPY Majority vote decision.

**Aggregation:** the final page closure policy decision can also be determined by comparing the aggregation of the all the counters values, Equation (1), against a certain threshold, Equation (2).

$$\begin{aligned} \text{if } \sum \text{AddressBitCounters} < \text{Threshold} &\rightarrow \text{Open Page} \\ \text{else} &\rightarrow \text{Close Page} \end{aligned} \quad (1)$$

$$\text{Threshold} = \frac{\text{AddrBitsWidth} \times \text{CounterValue}}{2} \quad (2)$$

The experimental results show similar results using either majority or aggregation. Thus only the majority vote decision scheme is used in the final experimental results.

### 3.3 HAPPY – Time-based Prediction

As a case study to show how HAPPY can be applied to a time-based prediction algorithm we chose the Intel-adaptive open-page policy [9, 21], for example, used by the Intel Xeon X5650 [11]. The basic structure of such a page closure policy is presented in Figure 6.

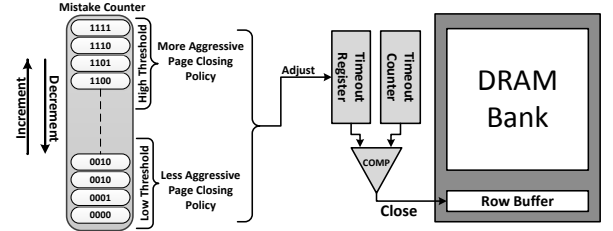


Figure 6: Intel-adaptive page policy predictor basic structure.

The integrated memory controller used in this Intel processor can be configured at boot time to employ one of the following three different page closure policy schemes: close-page, fixed-open and Intel-adaptive open page. The *fixed-open page policy* keeps a row open for a fixed period of time and closes it after that. The Intel-adaptive scheme is an advanced version of the fixed-open schemes. Similar to the fixed-open policy, in this structure, each row buffer within a bank has a Timeout Counter (TC) and a Timeout Register (TR). A row will be kept open until TC reaches the TR and then closed. However, the initial TR might not be a suitable value for all the benchmarks. Thus, the Intel-adaptive scheme provides a technique to update the TR at run time using a 4-bit Mistake Counter (MC). Whenever a page conflict happens that could have been a page-empty, since there was enough time to precharge the last accessed row, then the MC is decremented. Whenever a page empty could have been a page-hit, since the row being accessed is the same as the last accessed row in that bank, then the MC is incremented. After a specific time interval the MC will be checked against a pre-predefined low and high threshold to see if either a less or more aggressive page closure policy is required. If the MC is higher than the high-threshold then the TR will be incremented to keep the accessed row open for a longer period and if the MC is lower than the low-threshold the TR will be decremented to close the accessed row sooner.

Figure 7 depicts the HAPPY implementation of the Intel-adaptive open page policy. This time the aim is to extract the timeout value for each row to be kept open from the physical address bits. We use the same methodology explained in the previous section; the only difference is that instead of using simple saturating counters a Monitor Unit is dedicated to each physical address bit location. Each monitoring unit includes a MC and a TR with the same function as the original implementation of Intel-adaptive page policy. A global timeout counter is still required to keep track of row closing and opening times on a per bank basis.

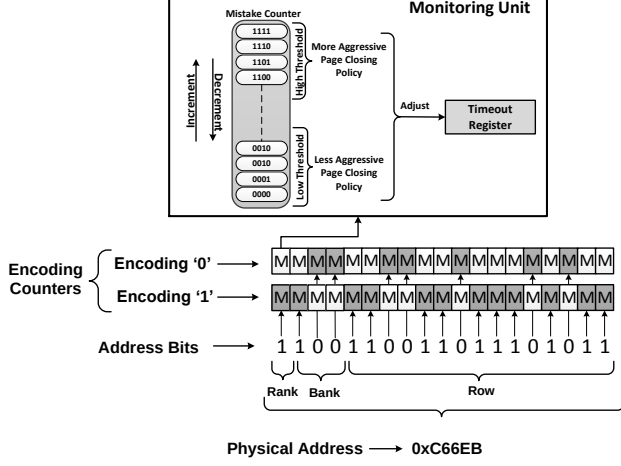


Figure 7: HAPPY implementation of Intel-adaptive page policy predictor.

Updating the MCs is as before but this time it is applied per physical address bit basis rather than per bank. Moreover, instead of having a global timeout register per bank, the time period that a row can be kept open will be calculated from the aggregation of all the participant bits for an accessed physical address.

To make a fair comparison between HAPPY and original implementation of this page policy, the size of the MC is chosen as before (e.g. 4-bit) and the size of TR in each physical bit location is chosen small enough that the maximum time that a row can be kept open is equal to having one global TR per bank.

### 3.4 HAPPY – Intuition

**Observation:** The main intuition behind HAPPY is based on the observation that addresses that are spatially close together tend to have a similar page-closure policy preference. HAPPY is devised to exploit such behavior by fine grain monitoring of physical address bits behavior.

**Ensemble Methods:** Although HAPPY was developed by analysis and observation of the experiments, we believe that there are Machine Learning principles that can justify the intuition behind HAPPY. A mathematical/theoretical framework that can explain HAPPY is that of Ensemble Methods [6]. The family of algorithms categorized as Ensemble methods combines multiple (normally simple) predictors. The theory explains how combining such predictors, it can be obtained a much improved predictor provided certain diversity properties among the predictors are met. Random forest, and neural networks are examples of very successful prediction algorithms part of the ensemble family. This pa-

per addresses an online learning scenario and uses a fixed number of predictors with a non-linear combination function. When applying our technique to Intel-adaptive, we generate a pair of simple predictors per physical address bit and solve a regression problem. Each pair of predictors is trained using a different single physical bit (different features) and each member of the pair is trained using only Zero or One occurrences (different dataset); mechanisms that can improve diversity. In Happy, we use two counters per physical address bits; e.g. 4GB is represented with 38 counters. If we were to limit ourselves to only a binary decision, then the maximum number of possible decisions that can be stored would be 2 to the power of 38. If we increase to 8GB, then we use 40 counters, and thus the maximum number of possible decisions that can be stored would be 2 to the power of 40. Thus as we increase the size of memory we are also increasing the number of possible decisions representable.

## 4. EVALUATION METHODOLOGY

Page closure prediction algorithms for DRAMs are sensitive to the application memory access patterns. To address this, we carry out an extensive evaluation, described as follows:

**Simulator:** USIMM [7], a detailed memory system simulator, is used as our main simulation platform. We extended USIMM to support five different, existing page closure policies (i.e. Open-Page, Close-Page, Hybrid, Fixed-Open and Intel-adaptive open page) plus the two implementations of HAPPY which are described in Section 3 (i.e. Hybrid-HAPPY and Intel-adaptive-HAPPY). The scheduling algorithm in the memory controller is FR-FCFS; first ready, first come first serve. We evaluated HAPPY based on different memory configurations, 2 GB for single-thread and 4 GB for multi-thread workloads. To increase the memory congestion we configured USIMM with 1 channel and 1 rank. The baseline USIMM system configuration parameters are captured in Table 2.

| Processor          |                       |
|--------------------|-----------------------|
| Clock Speed        | 3.2GHz                |
| Pipeline depth     | 10                    |
| ROB size           | 32                    |
| DRAM Parameters    |                       |
| DRAM Size          | 2 - 4 GB              |
| Bus Speed          | 800MHz                |
| Configuration      | 1Channel,1Rank,8Banks |
| Row per bank       | 65,536                |
| Cache line per row | 128                   |

Table 2: Simulation paramteres.

**Address Mapping Schemes:** The number of page conflicts in DRAMs and as a result the memory performance is susceptible to the memory address mapping scheme. The experiments consider 3 different address mappings presented in Figure 8. The first mapping 1 is a standard mapping to maximize row buffer locality. The next two address interleaving policies are state-of-the-art schemes proposed by Kaseridis *et al.* [14] and Zhang *et al.* [27]. The proposed mapping by Zhang *et al.* [27] XORs part of the row address bits with the banks address bits to produce a new bank index (see Figure 8b). Kaseridis *et al.* [14] extend this technique by

producing the column index using different section of physical address bits (Figure 8c). Both techniques aim to reduce page conflicts in DRAMS. Our experiments shows that the minimalist open page policy (Mapping 3) performs better for most of the workloads. Therefore, this address mapping scheme is employed in all the experiments. Focusing on the best page closure policy (i.e. Intel-Adaptive-HAPPY), we also report the sensitivity of HAPPY for all the three address mapping schemes.

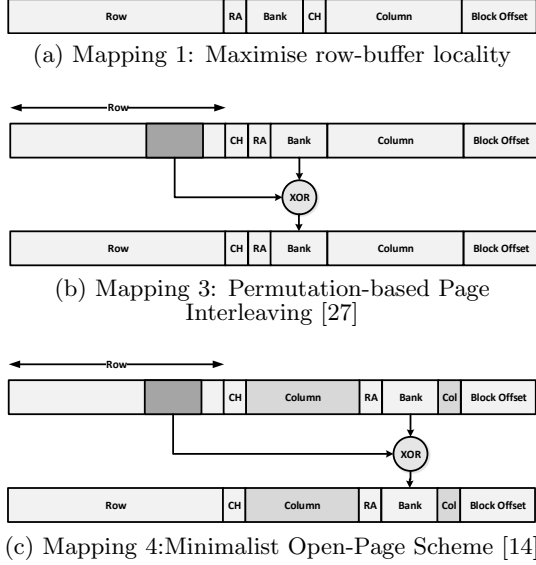


Figure 8: Different Address Interleaving schemes.

**Workloads:** The workloads include a wide range of memory intensive applications (i.e. 48 workloads) from different benchmark suites (PARSEC [4], SPEC [8], BIOBENCH [2], HPC and COMMERCIAL) and representative regions of interest for each application. Table 3 lists the workloads and their corresponding benchmark suites.

The USIMM simulator can run arbitrary multi-application workloads using multiple traces. To increase the variety of memory access patterns, we set up USIMM for multi-applications to produce 22 random workload mixes; a combination of 4-thread, 8-thread and 16-thread applications. Table 4 listed these multi-core workloads considering the prefix of single thread workloads presented in Table 3. Overall the experiments consider 70 workload mixes.

**Memory Footprint (MF):** To evaluate the performance of page closure predictors a careful study has to be carried out otherwise the performance and accuracy numbers might be misleading. For instance, if an application targets a very small portion of memory then it might be possible to predict its behavior using very small number of performance counter whereas if the application accesses all over of the memory space then it might be more difficult to keep track of the application access pattern with only a few counters (e.g. HAPPY). To have a fair evaluation methodology we made sure that the memory traces cover a wide range of access pattern. To this aim, we monitored the total physical pages accessed (Memory Footprint) per application at run time and we can confirm that our single thread applications have the average MF of 30% (up to 97%), our 4-thread workloads have the average MF of 50% (up to 75%), our 8-

thread workloads have the average MF of 70% (up to 85%) and our 16-thread workloads have the average MF of 95% (up to 99.8%).

## 5. RESULTS AND DISCUSSIONS

This section analyzes the different page closure policy prediction schemes compared with using HAPPY by looking at execution time, accuracy and scalability. Hereafter, The HAPPY implementation of Hybrid page policy is called Hybrid-HAPPY for brevity. Similarly, The HAPPY implementation of Intel-adaptive open-page policy is called Intel-adaptive-HAPPY. The results in Figure 10 and Figure 11-14 are normalized to the ‘static profiling’; the lower the bar the better the performance.

### 5.1 Prediction Accuracy

Understanding the prediction accuracy for the different types of page closure predictors has its pitfalls. For instance, prediction accuracy in the case of Hybrid predictors is straight forward as the prediction outcome is either opening or closing a page (binary classification). However, in the case of the Intel-adaptive technique the accuracy needs to be described based on the timeout value (regression). Consider a scenario where a page has to be open for 40 clock cycles to get a page hit and Intel-adaptive predicts 39 clock cycles. In this case the prediction accuracy should be calculated differently.

To have a fair evaluation across all the predictors with a different nature of prediction, we calculate the prediction accuracy based on the Page-Hit and the Page-Miss prediction outcome. In fact, the main purpose of using page policy predictors is to increase the page hits and reduce the page misses in the DRAM. Thus, we calculate the *Oracle page-hits* (maximum possible page-hits when having a perfect predictor) and *Oracle page-misses* (minimum possible page-misses when having a perfect predictor) and evaluate the actual page-hits and page-misses occurred during execution time of each workload against the oracle numbers. Figure 9 presents the prediction accuracy (GMEAN) of different predictors across all the workloads evaluated. The open-page and close-page policies deliver the maximum prediction accuracy for page-hits and page-misses, respectively. This happens because an open-page policy leaves all the page open and then it can cover all the possible page-hits in the system and non of the page misses. A close-page policy behaves similarly but in the opposite scenario. The hybrid-page policy delivers a moderate page-miss and page-hit prediction accuracy (around 60%). The Intel-adaptive and fixed-open both deliver a good prediction accuracy for both page-hits (80% and 75.8%) and page-misses (83.5% and 90.4%) respectively. Overall, the HAPPY implementation of both Intel-adaptive and hybrid are slightly more accurate than the original implementation. This prediction accuracy numbers justify the execution times presented in Figure 10. Also, from the accuracy results, it can be concluded that the page-hit prediction accuracy has a higher impact on the overall execution time than the page-miss prediction accuracy.

### 5.2 Performance Analysis

Figure 10 summarizes the performance of different prediction algorithms, normalized to the ‘Static Profiling’, for all the benchmarks. Each bargraph in this figure represents the Geometrical Mean (GMEAN) of the execution time for

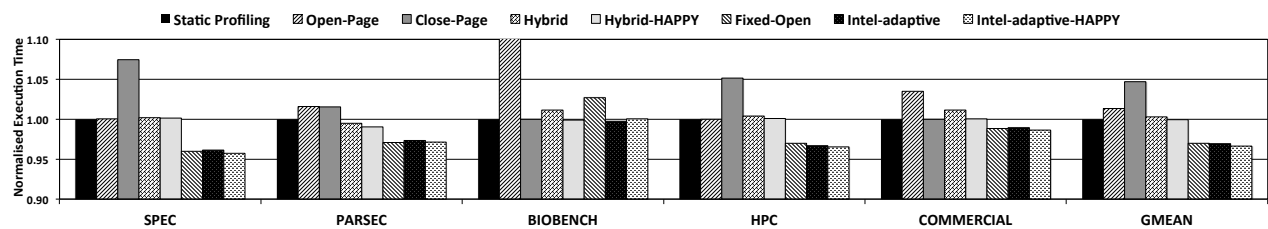


Figure 10: Average relative performance to static profiling for all the single-thread workloads.

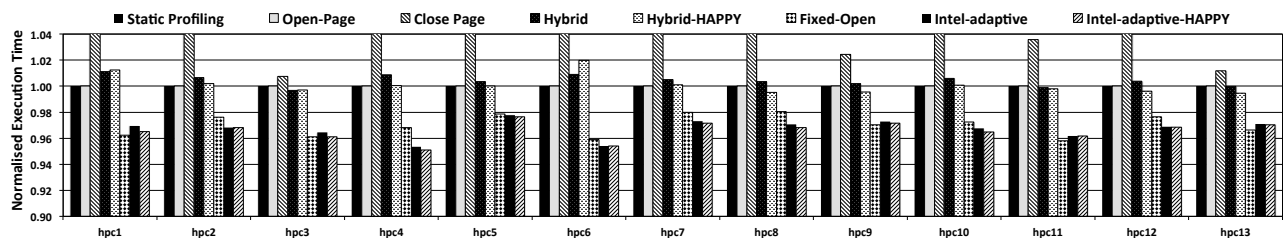


Figure 11: Relative performance to static profiling for HPC workloads.

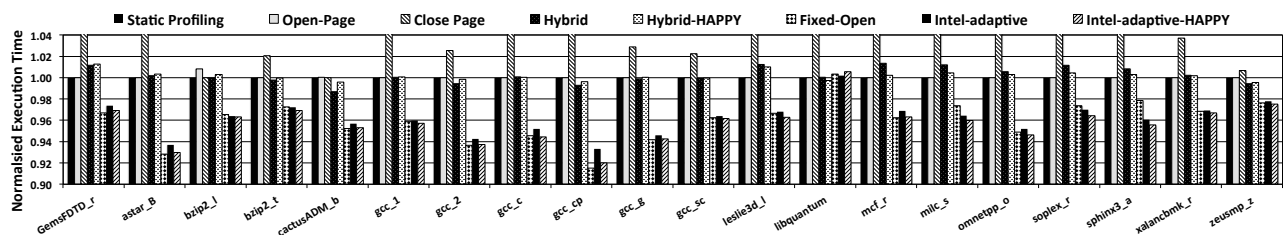


Figure 12: Relative performance to static profiling for SPEC workloads.

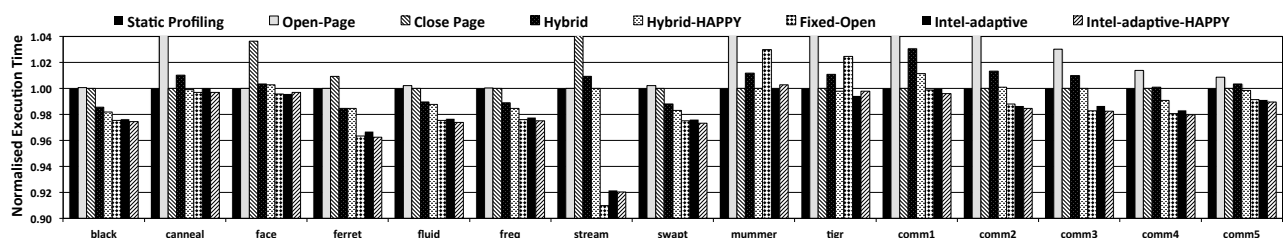


Figure 13: Relative performance to static profiling for PARSEC, BIOBENCH and Commercial workloads.

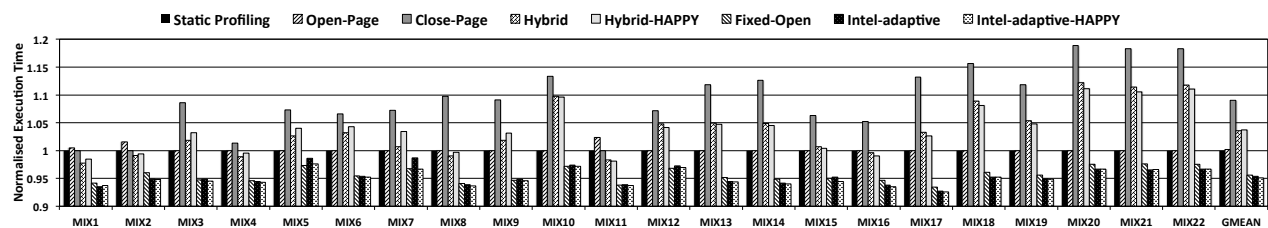


Figure 14: Relative performance to static profiling for Multi-Core workloads.

| Benchmark Suites |               |                   |                 |
|------------------|---------------|-------------------|-----------------|
| SPEC             |               | PARSEC            | COMMERCIAL      |
| (a) GemsFDTD_r   | (k) astar_B   | (u) canneal       | (D1) comm1      |
| (b) bzip2_l      | (l) bzip2_t   | (v) streamcluster | (D2) comm2      |
| (c) cactusADM_b  | (m) gcc_l     | (w) blackscholes  | (D3) comm3      |
| (d) gcc_2        | (n) gcc_c     | (x) facesim       | (D4) comm4      |
| (e) gcc_cp       | (o) gcc_g     | (y) ferret        | (D5) comm5      |
| (f) gcc_sc       | (p) mcf_r     | (z) fluidanimate  | <b>BIOBENCH</b> |
| (g) milc_s       | (q) omnetpp_o | (A) freqmine      | (E) mummer      |
| (h) soplex_r     | (r) sphinx3_a | (B) swaption      | (F) tigr        |
| (i) xalancbmk_r  | (s) zeusmp_z  | HPC               |                 |
| (j) libquantum   | (t) leslie    | (C) hpc1 - hpc13  |                 |

Table 3: Standard workloads and benchmark suites.

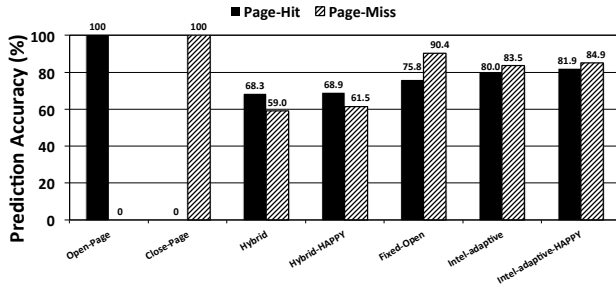


Figure 9: Prediction accuracy for different predictors.

the number of running workloads for each category. The detailed performance of the prediction algorithms for individual workloads is presented in Figures 11–13. These figures again confirm that a static page closure policy cannot deliver the optimum execution time for all the workloads. The corresponding workloads for HPC and SPEC benchmarks mostly prefer open-page policy. On the other hand, the corresponding workloads for PARSEC, BIOBENCH and COMMERCIAL workloads mostly prefer open-page policy.

**Overview:** Our experimental results show that the best page closure prediction scheme (i.e. Intel-adaptive-HAPPY) delivers 5% and 8% better average performance across all the workloads (up to 12% and 20%) in comparison with open-page and close-page policy respectively. Overall, the HAPPY implementation of both Hybrid and Intel-adaptive achieved similar performance when compared with the original implementation of these page closure policies albeit with a much lower hardware overhead. Comparing the Intel-adaptive with the Intel-adaptive-HAPPY page policy shows that the HAPPY implementation can reduce the cost of implementation by 5× for the evaluated 64 GB memory size (up to 40× for a memory size of 512 GB) while improving the performance up to 2%. Similar behavior can be observed for Hybrid and Hybrid-HAPPY. Hybrid-HAPPY shows 182,000× reduction in cost of implementation for the evaluated 64 GB memory size (up to 1.2M× for memory size of 512 GB) while improving the performance up to 3% for some workloads.

Similarly, as Figure 14 presents, for multi-thread applications, even with a very high memory footprint, HAPPY performance is consistent and it delivers similar performance to the original implementation. The experimental results show

that Intel-adaptive-HAPPY delivers 5% and 14% better average performance across all the workloads (up to 9% and 22%) in comparison with open-page and close-page policy respectively.

**Sensitivity to Address Mapping Schemes:** To investigate the sensitivity of HAPPY to different address mappings we select the best page closure policy (i.e. Intel-adaptive-HAPPY) across all the predictors presented in this paper and evaluate it with the three address mappings presented in Figure 8. Figures 15 and 16 illustrate the prediction accuracy of Intel-adaptive (original and HAPPY implementation) using the different mapping schemes. These results show that the HAPPY implementation of Intel-adaptive always delivers identical or slightly better results than the original implementation no matter which address mapping is used.

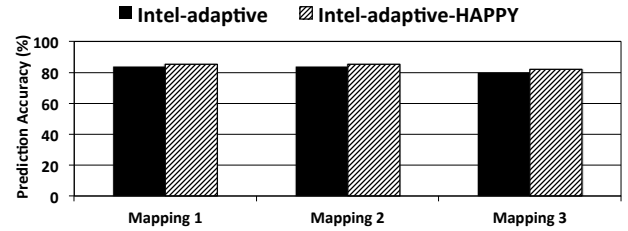


Figure 15: Page-hit prediction accuracy with different address mappings.

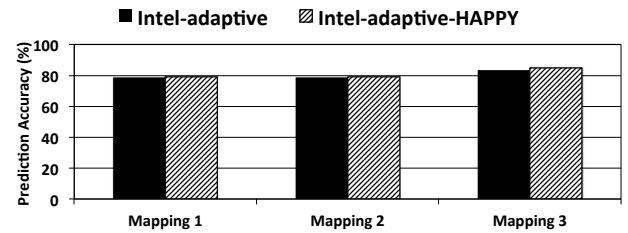


Figure 16: Page-miss prediction accuracy with different address mappings.

### 5.3 Sensitivity to Memory Size

We have evaluated HAPPY for up to 64 GB DRAM size and the results shows that HAPPY has a consistent behavior

| Multi-Core Workloads  |                                                           |
|-----------------------|-----------------------------------------------------------|
| MIX1: (w-D3-D3-F)     | MIX12: (b-n-s-w)                                          |
| MIX2: (D1-D5-E-F)     | MIX13: (w-D1-D5-x-y-z-t-F)                                |
| MIX3: (D1-x-y-F)      | MIX14: (D1-D4-D5-x-z-x-B-F)                               |
| MIX4: (D2-D4-A-F)     | MIX15: (D1-D4-D5-j-E-D5-v-F)                              |
| MIX5: (D2-D4-j-E)     | MIX16: (D2-D4-D5-z-A-A-D4-F)                              |
| MIX6: (D2-y-t-F)      | MIX17: (C5-C6-u-l-e-o-p-h)                                |
| MIX7: (D4-D5-g-F)     | MIX18: (C13-C14C17-C18-C21-C2-C4-v-k-l-c-m-e-n-h-s)       |
| MIX8: (D4-x-x-F)      | MIX19: (C13-C18-C21-C2-C6-u-v-C21-u-l-l-o-t-p-h-s)        |
| MIX9: (x-y-z-F)       | MIX20: (C14-C17-C21-C22-C2-C4-C5-C8-C14-C21-C4-k-e-o-a-p) |
| MIX10: (C21-C22-C4-b) | MIX21: (C17-C21-u-C17-q-q-i-t-o-b-o-a-t-p-q-i)            |
| MIX11: (C5-C6-u-e)    | MIX22: (C18-C22-C5-C6-C8-u-k-l-d-e-n-o-p-q-h-r)           |

Table 4: Randomly generated Multi-Core workloads.

ior as the memory size increase. Our experimental results suggests that the effective factor on HAPPY performance is the utilization of memory address space rather than the size of memory. For this reason, we considered a 4 GB memory organization with up to 99.8% memory space utilization for our multi-thread experiments (results are presented in Figure 14). Even in this situation, the results show that HAPPY delivers a competitive performance against the original implementation of both Hybrid and Intel-adaptive page policies while reduces the hardware overhead significantly.

## 5.4 Scalability with Memory Size

Figure 17 depicts the required storage (bytes) for each prediction algorithm for different sizes of memory. The HAPPY implementation of the hybrid prediction technique is orders of magnitude (e.g. up to  $1.2M\times$ ) cheaper than the original implementation while we show that it delivers similar performance to that of the original implementation. In the case of Intel-adaptive page closure policy, the HAPPY implementation requires slightly more resources than the original implementation for memory sizes of less than 8 GB. However, as the memory size grows, the Intel-adaptive-HAPPY outperforms the scalability over the original implementation up to  $40\times$  for a memory size of 512 GB. Table 5 depicts the required performance counters for different page closure policies with and without HAPPY considering a memory system with X channels, Y ranks, Z banks and W rows.

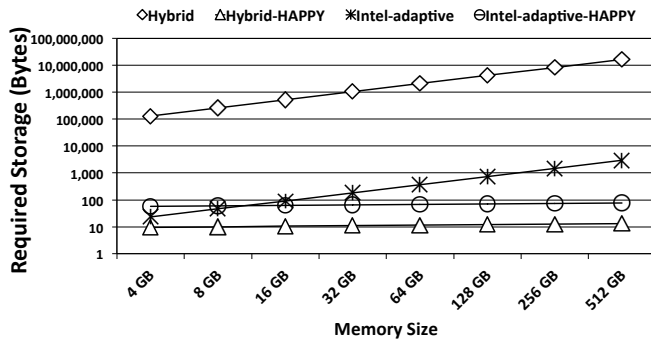


Figure 17: Scalability of different page closure prediction algorithms.

| Implementation | Required Counters                                      |
|----------------|--------------------------------------------------------|
| Hybrid         | $X \times Y \times Z \times W$                         |
| Hybrid-HAPPY   | $(\log_2 X + \log_2 Y + \log_2 Z + \log_2 W) \times 2$ |
| Intel-adaptive | $(X \times Y \times Z) \times 2$                       |
| Intel-HAPPY    | $(\log_2 X + \log_2 Y + \log_2 Z + \log_2 W) \times 4$ |

Table 5: Required performance counters for different page closure policies.

## 5.5 Prediction Algorithms - Pros & Cons

Due to the nature of the implementation and structure of each predictor, each of them might or might not work in a specific situation. Here, we discuss such situations.

**Static Policies:** the open-page policy works best for high locality workloads but degrades the performance of DRAMs significantly for the workloads with highly random or dynamic memory accesses. The close-page policy has the completely opposite behavior. PARSEC and SPEC workloads are good examples which show the different behavior of open-page and close-page policy.

**Fixed-Open:** The performance of this type of algorithms is fairly susceptible to its predefined timeout value. Similarly to the methodology presented in [14], we select this value to be equal to  $t_{RC}$ , that is the minimum time limitation between consecutive accesses to different rows within a bank, in the experiments. This time delay provides enough opportunity to capture a possible page hit; cf. the results presented in Figure 10. However, for non-memory intensive threads with high locality or memory intensive with low locality (e.g. ‘mummer’ and ‘tigr’) this technique might not work well. The reason is that, for the first category, the time interval between memory requests might be higher than the fixed timeout value which means this technique will close the row before a page hit happens. Similarly, for the second category the time interval between memory requests might be lower than the fixed timeout value, which means that a row would be kept open for an unnecessary time which, most likely, would lead to other page conflicts.

**Hybrid:** the integrated saturating counters employed in this category (either the original or HAPPY implementation) train by the number of page-hits and page-conflicts that they face. Therefore, the prediction accuracy of these types of techniques is fairly sensitive to the distribution of page hits/conflicts within DRAMs. For instance, ‘stream-cluster’ presents such a behavior.

**Intel-adaptive:** our experiments show that this predic-

tion algorithm is the best across all the presented schemes in this paper. However, one weakness of this technique is the updating granularity of TR. In our experiments, every time that checking of the MC suggests a more or less aggressive page closure policy, TR is incremented or decremented by one respectively. Updating granularity by one step (increment or decrement) delivers a fine tuning of the TR but reduces the training rate of the overall prediction technique. This means that, for workloads where the application access pattern behavior changes frequently (e.g. between high and low locality accesses pattern) within different time phases, the Intel-adaptive scheme might not be able to deliver its best performance. Such behavior can be observed in ‘can-neal’ or ‘comm1’.

**HAPPY:** so far we have focused on the advantages of HAPPY. However, like all the other proposed techniques, HAPPY also has weaknesses. Considering the global nature of a HAPPY implementation it is expected that HAPPY cannot perform as efficiently as fine grain schemes for workloads with fairly dynamic behavior targeting a small part of DRAMs locally. This can be seen in workloads such as ‘tigr’.

## 5.6 Flexibility

HAPPY is the proof of the concept that the physical address bits can be the source of useful information that can be extracted using the right encoding and decoding techniques. This makes HAPPY a fairly flexible tool that can be applied to different prediction algorithms that have not been practical due to the cost of implementation, making them feasible. In this paper we applied HAPPY to two completely different prediction schemes and showed how the performance and scalability of these scheme improved. However, page closure policies are not the only candidate that can take advantage of the knowledge presented by HAPPY.

## 6. RELATED WORK

Succinctly, prior research in this area can be categorized in two main groups: access-based and time-based techniques.

**Access-based techniques** are those that monitor and keep a history of the row hits and row misses at different granularity in DRAMs to make a prediction of the future page closure policy for each row or bank within a DRAM memory system. Xu *et al.* [26] proposed a two-level dynamic SDRAM policy predictor which collects the row hit/miss behavior for the last  $n$  accesses in a history register. For each entry in the history register, there is a 2-bit saturating counter that keeps track of the page closure policy for each access. Huan *et al.* [10] proposed the Processor-Directed dynamic page policy where the processor keeps track of the last row access to each bank to predict page hits or misses for future memory requests. The processor sends this information to the memory controller to specify the page closure for the next memory access. Awashti *et al.* [3] keep track in a history table of the number of accesses each row has before closing it. When a row is open the number of expected accesses to that row is looked up, if there is no recorded entry for the accessed row in the history table that row is kept open. However if there is an entry for the accessed row it will be closed after the expected number of accesses suggested by the history table. More techniques using access-based page closure prediction can be found in [20, 16, 17, 23, 25, 24, 22].

**Time-based techniques** mainly focus on predicting the optimum time that a row can be left open. Blackmore [5] presented a quantitative analysis of page closure predictors. This work specifically focused on the Intel-adaptive page policy structure and tried to improve it by introducing the inter-arrival distribution concept. Stonkovic *et al.* [25] used the concept of *live-time* and *dead-time* to predict the page closure. The live-time is the time interval between opening a row until the last access to that row while dead-time is the interval from the last access to an open row until its closing. If the predictor predicts a zero live-time or if it predicts that the row has entered its dead-time period, then the row will be closed immediately after the DRAM access otherwise it will be kept open for future accesses. In another work, Kaseridis *et al.* [14] used the concept that in DRAMs there is a minimum time limitation of  $t_{RC}$  between two activations within a bank and speculatively leave the pages open for the  $t_{RC}$  period.

To sum up, HAPPY is the only technique which considers an encoding based on the memory address bits offering a compact means of storing history to inform the predictions. In addition, we have shown how to apply HAPPY to Time-based (i.e. Intel Adaptive HAPPY) and Access-based techniques (i.e. Hybrid HAPPY).

## 7. CONCLUSIONS

DRAM performance is dependent on the memory access pattern and, more specifically, the number of page-hit and page-conflicts that occur at run time. Modern DRAM controllers employ advanced page closure policy predictors to improve performance by trying to transform page-conflicts into page-empty (e.g. by closing the last accessed row at the “right time”), and page-empty cases into page-hits (e.g. by keeping open the last accessed row for longer time). However the main challenge is to balance the prediction accuracy of these predictors with manageable hardware overheads (scalability) as we increase the size of DRAM.

We have described HAPPY – a compact and efficient binary-encoding technique – to alleviate the scalability problem of DRAM page closure predictors. HAPPY relies on the simple observation that there is a strong correlation between the physical address bits of memory addresses requested by processors and the internal structure of the DRAM as there is a fixed-address mapping scheme. Thus, the physical address bits carry the information that a memory controller needs to predict the page-hit or page-conflict for a particular access. Considering this, the required performance counters and monitoring units needed by the page closure prediction algorithms can be encoded from the physical address bits. Doubling the size of DRAM only implies one extra physical address bit. This means that with HAPPY only one extra monitoring unit is required to predict the DRAM page closure policy when the size of memory is doubled. In other words, HAPPY offers the smallest hardware overhead to implement dynamic DRAM page closure predictor algorithms.

We have evaluated HAPPY by applying it to a traditional Hybrid page closure policy, as well as the state-of-the-art Intel-adaptive open page policy included in Intel Xeon X5650. The experimental results show that the HAPPY implementation of Intel-adaptive page policy can reduce the cost of implementation by  $5\times$  for the evaluated 64 GB memory size (up to  $40\times$  for a memory size of 512 GB) while maintaining the prediction accuracy. The other case study

shows 182,000 $\times$  reduction in cost of implementation for the evaluated 64 GB memory size (up to 1.2M $\times$  for memory size of 512 GB) while maintaining the prediction accuracy. The experiments have also reported the accuracy of the predictors and have studied the sensitivity towards the memory address-mapping. In both scenarios, HAPPY maintains its key advantage of offering no degradation of prediction accuracy while reducing significantly the hardware overhead.

## 8. ACKNOWLEDGMENTS

The research leading to these results has received funding from the EPSRC programme grant PAMELA EP/K008730/1 and the European Union's Seventh Framework Programme (FP7/2007-2013) under grant agreement no. FP7-318633. Luján is supported by a Royal Society University Research Fellowship.

## 9. REFERENCES

- [1] John Ousterhout. Ramcloud.
- [2] K. Albayraktaroglu, A. Jaleel, X. Wu, M. Franklin, B. Jacob, C.-W. Tseng, and D. Yeung. Biobench: A benchmark suite of bioinformatics applications. In *Performance Analysis of Systems and Software, 2005. ISPASS 2005. IEEE International Symposium on*, pages 2–9. IEEE, 2005.
- [3] M. Awasthi, D. W. Nellans, R. Balasubramonian, and A. Davis. Prediction based dram row-buffer management in the many-core era. In *Parallel Architectures and Compilation Techniques (PACT), 2011 International Conference on*, pages 183–184. IEEE, 2011.
- [4] C. Bienia, S. Kumar, J. P. Singh, and K. Li. The parsec benchmark suite: characterization and architectural implications. In *Proceedings of the 17th international conference on Parallel architectures and compilation techniques*, pages 72–81. ACM, 2008.
- [5] M. Blackmore. A quantitative analysis of memory controller page policies. *Notes*, 2013:01–01, 2013.
- [6] G. Brown. Ensemble learning. *Encyclopedia of Machine Learning*, 2010.
- [7] N. Chatterjee, R. Balasubramonian, M. Shevgoor, S. Pugsley, A. Udiipi, A. Shafiee, K. Sudan, M. Awasthi, and Z. Chishti. Usimm: the utah simulated memory module. *University of Utah, Tech. Rep*, 2012.
- [8] K. M. Dixit. The spec benchmarks. *Parallel computing*, 17(10):1195–1209, 1991.
- [9] J. Dodd. Adaptive page management, July 11 2006. US Patent 7,076,617.
- [10] D. Huan, Z. Li, W. Hu, and Z. Liu. Processor directed dynamic page policy. In *Advances in Computer Systems Architecture*, pages 109–122. Springer, 2006.
- [11] Intel. Intel xeon processor x5650.
- [12] K. Itoh. *VLSI memory chip design*, volume 5. Springer New York, 2001.
- [13] B. Jacob, S. Ng, and D. Wang. *Memory systems: cache, DRAM, disk*. Morgan Kaufmann, 2010.
- [14] D. Kaseridis, J. Stuecheli, and L. K. John. Minimalist open-page: A dram page-mode scheduling policy for the many-core era. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 24–35. ACM, 2011.
- [15] B. Keeth. *DRAM circuit design: fundamental and high-speed topics*, volume 13. Wiley. com, 2008.
- [16] C. Ma and S. Chen. A dram precharge policy based on address analysis. In *Digital System Design Architectures, Methods and Tools, 2007. DSD 2007. 10th Euromicro Conference on*, pages 244–248. IEEE, 2007.
- [17] S. Miura, K. Ayukawa, and T. Watanabe. A dynamic-sdram-mode-control scheme for low-power systems with a 32-bit risc cpu. In *Proceedings of the 2001 international symposium on Low power electronics and design*, pages 358–363. ACM, 2001.
- [18] D. Ongaro, S. M. Rumble, R. Stutsman, J. Ousterhout, and M. Rosenblum. Fast crash recovery in ramcloud. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, pages 29–41. ACM, 2011.
- [19] J. Ousterhout, P. Agrawal, D. Erickson, C. Kozyrakis, J. Leverich, D. Mazières, S. Mitra, A. Narayanan, G. Parulkar, M. Rosenblum, et al. The case for ramclouds: scalable high-performance storage entirely in dram. *ACM SIGOPS Operating Systems Review*, 43(4):92–105, 2010.
- [20] S.-I. Park and I.-C. Park. History-based memory mode prediction for improving memory performance. In *Circuits and Systems, 2003. ISCAS'03. Proceedings of the 2003 International Symposium on*, volume 5, pages V–185. IEEE, 2003.
- [21] Rajinder Gill. Everything you always wanted to know about sdram memory but were afraid to ask.
- [22] R. C. Schumann. Design of the 21174 memory controller for digital personal workstations. *Digital Technical Journal*, 9:57–70, 1997.
- [23] V. Stanković and N. Milenković. Access latency reduction in contemporary dram memories. *Facta universitatis-series: Electronics and Energetics*, 17(1):81–97, 2004.
- [24] V. Stankovic and N. Milenkovic. Dram controller with a complete predictor: Preliminary results. In *Telecommunications in Modern Satellite, Cable and Broadcasting Services, 2005. 7th International Conference on*, volume 2, pages 593–596. IEEE, 2005.
- [25] V. V. Stankovic and N. Z. Milenkovic. Dram controller with a close-page predictor. In *Computer as a Tool, 2005. EUROCON 2005. The International Conference on*, volume 1, pages 693–696. IEEE, 2005.
- [26] Y. Xu, A. S. Agarwal, and B. T. Davis. Prediction in dynamic sdram controller policies. In *Embedded Computer Systems: Architectures, Modeling, and Simulation*, pages 128–138. Springer, 2009.
- [27] Z. Zhang, Z. Zhu, and X. Zhang. A permutation-based page interleaving scheme to reduce row-buffer conflicts and exploit data locality. In *Proceedings of the 33rd annual ACM/IEEE international symposium on Microarchitecture*, pages 32–41. ACM, 2000.